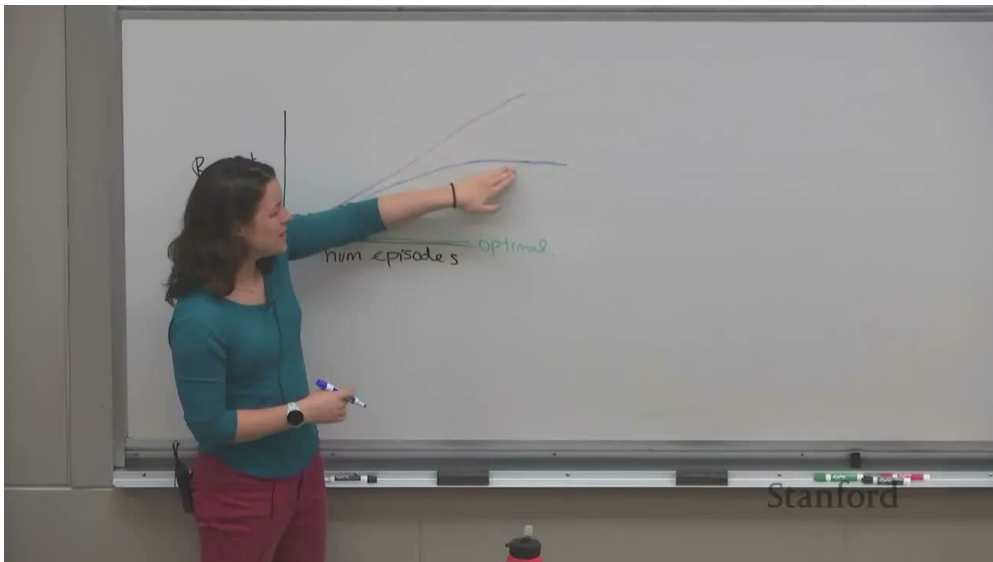


课程笔记

CS224R 第 14 讲: Exploration & Meta-Exploration

Codex 基于 Stanford CS224R 2025 春季课程视频、字幕与官方 slides 重写

2026-04-13



视频作者/频道: Stanford Online

发布日期: 2025 年 12 月 08 日

视频时长: 1:12:42

视频链接: <https://www.youtube.com/watch?v=4t1SKdi8teU>

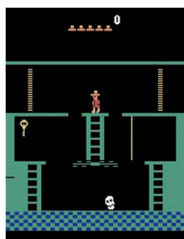
目录

1	为什么探索如此困难	2
1.1	本章小结	2
2	Bandit 视角：如何定义“好的探索”	3
2.1	本章小结	4
3	从 Bandit 到机器人与大模型	5
3.1	本章小结	5
4	测试时探索与 Meta-Exploration	5
4.1	本章小结	7
5	Meta-RL 在教育中的应用	7
5.1	本章小结	8
6	总结与延伸	8

1 为什么探索如此困难

这一讲开篇没有直接讲算法，而是先建立“探索问题到底难在哪里”的直觉。讲者用 Atari 里的 *Breakout* 与 *Montezuma's Revenge* 对比说明：在前者中，奖励频繁且行为-结果关系直观，标准 RL 往往能学得不错；而在后者中，关键奖励事件稀疏、延迟长、局部动作并不直接显出价值，因此“从零发现高回报行为链条”非常困难。

Montezuma's revenge



- Getting key = reward
- Opening door = reward
- Getting killed by skull = nothing (is it good? bad?)
- Finishing the game only weakly correlates with rewarding events
- We know what to do because we **understand** what these sprites mean!

Slide adapted from Sergey Levine

图 1: Montezuma's Revenge 例子说明：拿钥匙、开门等关键事件都很稀疏，算法并不知道哪些 sprite 值得关注。

课程随后引入 Mao 这个“规则只能靠试错发现”的牌类游戏，让学生站到算法视角去想：如果你只知道“犯错会受罚”，却不知道规则是什么，也不知道惩罚的语义是否合理，那么任何长时程任务都会变得异常艰难。任务越长、规则越隐蔽、奖励越稀疏，探索就越难。

这使得“探索”与“利用”看起来像两个定义：

- 探索：尝试尚未验证的新行为，希望发现更高回报；
- 利用：执行已知回报最高的行为。

但课程强调，这其实是同一个问题的两面。真正的挑战不是简单地“随机试更多动作”，而是在有限交互预算下，决定何时继续沿用当前最好策略、何时为未来收益而承担短期损失。

本讲的基本立场

探索问题的本质，不是“怎样更随机”，而是“怎样在不确定性下为未来信息价值付费”。

为了让直觉更接近日常，讲者给出三个典型例子：选餐馆、广告投放、油井勘探。这些例子都说明，一旦长期收益取决于今天是否愿意尝试新选项，探索与利用就不可分割。

1.1 本章小结

探索之所以难，不是因为“找不到随机性”，而是因为高回报往往需要长链条、稀疏反馈和对环境规则的推断。稀疏奖励只是表象，更深层的问题是不确定性与长时程信用分配。

2 Bandit 视角：如何定义“好的探索”

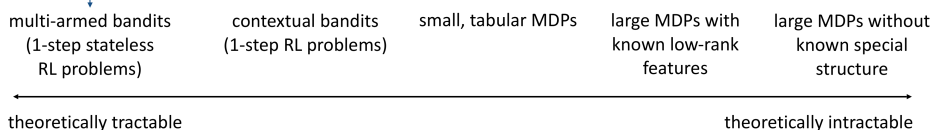
为了让讨论可分析，课程先退到最简单的 setting：multi-armed bandit。它只有单步 horizon、没有状态，但奖励是随机的，因此非常适合分析探索策略的优劣。

Exploration is hard

Can we derive an **optimal** exploration strategy?

what does optimal even mean?

Let's talk about these for a minute.



Slide adapted from Sergey Levine

图 2: 从 bandit 到大规模无结构 MDP，探索问题会迅速从可分析走向理论与计算上的困难。

在 bandit 里，课程引入了后续最重要的评价指标：regret。直观上，它衡量的是“如果我一开始就知道最佳动作，和我实际所选动作相比，累计少拿了多少回报”。

$$\text{Regret}(T) = T \max_a \mathbb{E}[r(a)] - \sum_{t=1}^T r(a_t).$$

这里：

- T 是累计交互轮数；
- a 表示任意候选动作；
- $\mathbb{E}[r(a)]$ 是动作 a 的期望回报；
- a_t 是第 t 轮实际采取的动作；
- $\text{Regret}(T)$ 越小，表示探索-利用权衡越好。

这个定义的关键之处在于，它不是只看最终找到没找到最优臂，而是把整个学习过程都纳入评价。一个算法如果一直随机选动作，regret 会近似线性增长；一个有效算法则希望 regret 以更慢的速度增长。

Exploration in Bandits

Optimism when uncertain
(e.g. upper confidence bound (UCB))

Track avg reward $\bar{r}_a$ for all actions

Exploit: pick $a_{\text{exploit}} = \arg \max_a \bar{r}_a$

Explore: pick $a_{\text{explore}} = \arg \max_a \bar{r}_a + C\sigma_a$
some kind of variance estimate

Intuition: assume unknown = good

Probability matching
(e.g. posterior sampling)

Assume $r(a_i) \sim p_{\psi_i}(r)$ for each arm a_i

Defines a POMDP with $\mathbf{s} := [\psi_1, \dots, \psi_n]$

1. Form beliefs $\hat{p}(\psi_1, \dots, \psi_n)$ from data
2. Sample an MDP $\psi_1, \dots, \psi_n \sim \hat{p}(\cdot)$
3. Take optimal action a assuming MDP is correct.

Intuition: iteratively improve model of bandit, act according to one model

These methods are widely used/studied in industry for recommenders, ad placement!

图 3: Bandit 中两条经典思路: UCB 的“在不确定时乐观”与 posterior sampling 的“按后验概率匹配”。

课程随后并列介绍两类经典策略:

- UCB: 对均值估计再加上不确定性 bonus, 优先尝试“可能比现在看起来更好”的臂。
- posterior sampling: 维护每个臂的后验分布, 从中采样一个世界模型, 再在该模型下选当前最优动作。

UCB 可以写成

$$a_t = \arg \max_a (\bar{r}_a + c \sigma_a).$$

这里:

- $\bar{r}_a$ 是动作 a 的当前平均回报估计;
- σ_a 是该动作的不确定性或方差估计;
- c 控制探索 bonus 的强度;
- a_t 是当前轮被选中的动作。

这个公式浓缩了 UCB 的全部直觉: 未知并不直接等于好, 但未知值得被加分, 因为它可能隐藏更高收益。

为什么 bandit 重要

它不是因为现实问题都像 bandit 那样简单, 而是因为这里能清楚定义 regret、比较算法、分析长期收益, 从而为更复杂的探索问题建立语言和直觉。

2.1 本章小结

Bandit 提供了探索问题最干净的实验台。通过 regret, 我们不再只关心“最终找没找到最优解”, 而是开始关心整个学习过程的累计损失; 通过 UCB 与 posterior sampling, 我们得到两种最经典的探索思想。

3 从 Bandit 到机器人与大模型

讲者紧接着提醒：不要误以为 bandit 里的优雅结论会自动延伸到大规模 MDP。一旦有状态、有长时程依赖、奖励稀疏且环境没有已知结构，探索很快就变成理论上和计算上都极难的问题。

课程在这一部分给了一个非常务实的结论。尽管学界尝试把 UCB、posterior sampling 等思想推广到机器人控制等大 MDP 中，但“从零探索”在现实大型问题里通常是不现实的。因此工业和前沿研究更常采用三类补救：

- 借助 demonstrations 或在 demonstrations 上预训练的基础模型；
- 尽量使用 shaped rewards；
- 让模型依赖更强的先验知识，而不是从纯随机交互中发现一切。

这也是为什么讲者会把机器人和 LLM 并列提起。大型语言模型之所以看起来“探索能力很强”，并不是因为它们在线上从零开始探索得更好，而是因为海量预训练已经把世界知识、语义先验和动作模式压进了参数里。某种意义上，它们是在用预训练替代在线探索。

3.1 本章小结

Bandit 给我们方法论，但现实中的机器人和 LLM 更多依赖先验、数据与奖励工程。真正困难的大规模探索问题，很少靠“从零试出来”解决。

4 测试时探索与 Meta-Exploration

在前半部分建立了探索基本图景之后，课程把话题拉回 Meta-RL。这里的问题变成：如果测试时代理必须先探索，再利用探索结果执行任务，那么该如何训练这种行为？

讲者先指出一个非常常见的 baseline：端到端优化探索 episode 与执行 episode 的总回报。这种做法原则上最直接，因为它直接对最终目标负责。

Recall: Why is End-to-End Training Hard for Meta-Learning with Exploration?

End-to-end approach: optimize exploration and execution episode behaviors end-to-end to maximize reward of execution



Coupling problem: learning exploration and execution depend on each other

—> can lead to poor local optima, poor sample efficiency

Liu, Raghunathan, Liang, Finn. *Decoupling Exploration and Exploitation for Meta-Reinforcement Learning without Sacrifices*. ICML 2021

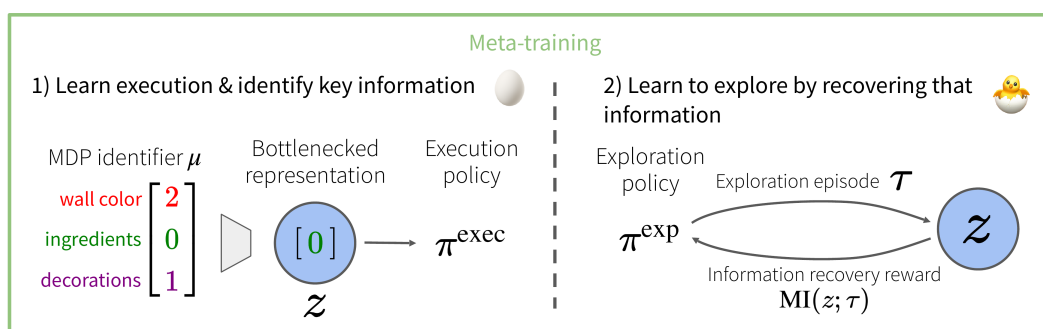
图 4: 元探索中的耦合问题：探索决定能否收集信息，执行决定信息能否转化为回报，两者相互依赖。

但它也有一个经典 failure mode: coupling problem。若代理不会探索，它就收集不到执行所需的信息；若它不会执行，即使探索对了也拿不到高回报。课程用“在新厨房先找食材再做饭”的例子说明，这会导致极差的样本效率和糟糕的局部最优。

这时可以回到上一讲提到的另一条路线：posterior sampling。PEARL 就是在元学习框架里显式建模任务潜变量后验，再按当前采样到的任务假设去行动。这种做法虽然不再是纯粹黑盒端到端，但它把“任务推断”和“策略执行”分成了两个更清晰的模块。

课程随后介绍 DREAM。它的核心思想不是直接逼网络同时学好探索与执行，而是先学会抽取与任务有关的信息表示，并通过 bottleneck 压掉无关信息，从而把探索与执行部分解耦。

Solution #3: Decouple by acquiring representation of task relevant information



Liu, Ragunathan, Liang, Finn. *Decoupling Exploration and Exploitation for Meta-Reinforcement Learning without Sacrifices*. ICML 2021

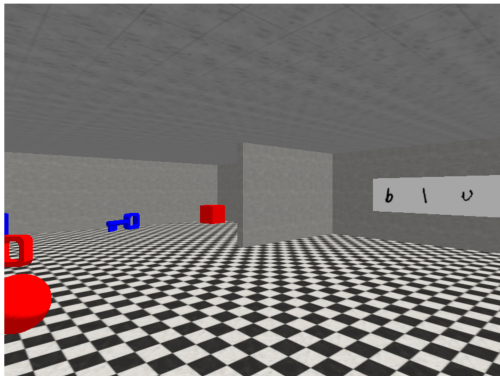
图 5: DREAM 的关键想法：先学习能识别任务差异的瓶颈表示，再让执行策略围绕该表示工作。

从 lecture 的叙述看，DREAM 的价值在于两点：

- 它保留了“测试时需要探索”的场景，而不是偷换成显式任务标签。
- 它把信息收集问题转化为“哪些信息对任务真正 relevant”的表示学习问题。

讲者还强调，DREAM 在理论上能在较弱假设下与端到端目标保持一致，在实践上则通常比完全耦合的端到端方法更高效。

Empirical Comparison: Sparse Reward 3D Visual Navigation Problem



- Task: go to the (key / block / ball), color specified by the sign
- Agent starts on other side of barrier, must walk around to read the sign
- Pixels observations (80 x 60 RGB)
- Sparse binary reward

Liu, Raghunathan, Liang, Finn. *Decoupling Exploration and Exploitation for Meta-Reinforcement Learning without Sacrifices*. ICML 2021

图 6: 在稀疏奖励 3D 导航问题上, DREAM 类方法需要先绕路读取线索, 再去目标物体位置。

本讲的研究判断

元探索真正难的不是“有没有任务记忆”, 而是“如何获得足以区分任务、又不和执行学习彼此拖累的信息”。这也是为什么表示学习和解耦成为关键。

4.1 本章小结

测试时探索把普通探索问题推进了一步: 不仅要发现高回报动作, 还要先发现“这个任务到底是什么”。端到端训练虽最完整, 却很容易耦合失效; PEARL 与 DREAM 则尝试通过结构化后验或信息瓶颈来缓解这一点。

5 Meta-RL 在教育中的应用

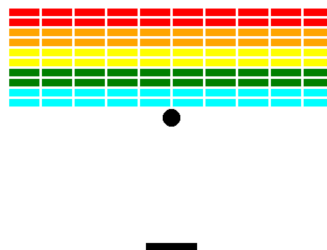
本讲最后给了一个非常具体也很有启发的应用: 把学生程序看作不同任务, 用 Meta-RL 代理在程序里“探索 bug”, 再据此生成反馈和辅助评分。

Example application: finding bugs & providing feedback in student programs

Bounce programming assignment
(Code.org)

```
Underlying env ID: 7540  
Env ID: 1  
Label: 1110000100101011  
Binary label: chemical-synthesis  
Action: None  
Reward: 0  
Coverage: 0  
Exploration reward: 0.020  
Prob: 0.006
```

Breakout assignment
(CS106A)



Time-consuming for instructors/TAs to give feedback, grades.
Use meta-RL to learn exploration!

图 7: 课程把学生程序视作不同任务: 代理先交互探索程序行为, 再定位 bug 并辅助反馈或评分。

Code.org 的 Bounce 和 Stanford CS106A 的 Breakout 都是很好的例子。不同学生提交的程序会有不同 bug, 有的球会被球拍“串住”, 有的会无限生成物体。教师或助教的工作, 本质上是通过交互去发现“这个程序的潜在规则和故障模式是什么”。这与 Meta-RL 的任务适应范式非常接近。

课程报告了两个很直接的收益:

- 在 Bounce 任务中, 系统能学会有针对性的探索行为, 主动触发关键事件来暴露 bug。
- 在 CS106A 2023 春季学期的评分试验中, 辅助系统带来了约 44% 更快、约 6% 更准确的 grading。

这部分的价值不只在结果数字, 而在于它说明 Meta-RL 并非只能用于导航、机器人或 bandit。只要存在“先探索行为, 再据此推断任务结构”的过程, Meta-RL 就可能成为一个合适的建模工具。

5.1 本章小结

教育应用把本讲的抽象问题落到了非常具体的交互系统上。它提醒我们: 探索不仅是找奖励, 也可以是为了更快地发现 bug、生成反馈和支持人类决策。

6 总结与延伸

这一讲分成两条主线。第一条主线是探索本身: 为什么稀疏奖励、长时程任务和规则不透明会让探索变难; 为什么 bandit 是研究探索的好起点; 为什么 UCB 与 posterior sampling 代表了两种经典范式。第二条主线是 meta-exploration: 如果测试时必须先探索当前任务, 再利用收集到的信息, 那么训练会遇到严重的耦合问题。

在研究判断上, 这讲非常克制。它没有声称“找到一种万能探索算法”, 反而一直在强调 problem structure 的重要性:

- 简单结构问题可以谈 regret 与近最优算法;
- 复杂大 MDP 往往依赖 demonstrations、预训练和 shaped rewards;

- 元探索则需要额外考虑任务识别、信息瓶颈和探索-执行解耦。

从课程脉络看，这讲实际上起到了桥梁作用。它一边把上一讲的 Meta-RL 进一步推到“测试时必须探索”的 setting，一边又为下一讲关于 autonomy、long-horizon task 和 hierarchy 埋下伏笔。对读者而言，最值得带走的不是某一个算法名字，而是这句隐含的工程判断：**探索难点的本质，常常取决于你缺的到底是动作尝试、任务识别，还是足够强的先验。**